

Multi-Agent Pathfinding (MAPF)

Scribe: Yuyan Wang, Kyle McDonald

2/10/2025

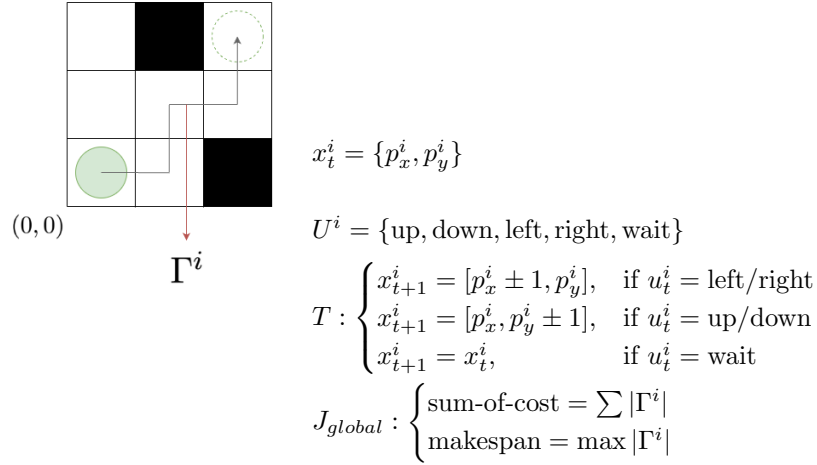
1 Introduction

Multi-Agent Pathfinding (MAPF) is the problem of computing conflict-free paths for multiple agents navigating a graph. It is one of the well-established subfields in robotics, with significant ongoing research. Notably, there is a dedicated MAPF workshop at AAAI, and the field gained mainstream recognition in 2005 following a paper by David Silver.

Interestingly, understanding MAPF requires recognizing that it is not inherently a robotics problem. The concept remains relevant even without physical robots, allowing for research and study purely in computational and algorithmic contexts. This raises the question: If MAPF is not fundamentally a robotics problem, then what is it?

2 Problem Definition

We will start by defining the classical MAPF problem:



Where

χ : the discrete state space (grid)

U^i : the discrete action space

T : the transition function

J_{global} : the global cost function

Classical MAPF problem is discrete, centralized, fully cooperative, communicative or non-communicative (design choice), and fully observable. Because it's centralized and cooperative, we have a global cost function. There are two main kinds of global cost function, sum-of-cost and makespan, where sum-of-cost is the sum

of the steps that all the agents in the system take and makespan is the number of steps for the last agent to reach the goal.

3 Objective of MAPF

The objective of MAPF is to find the trajectory $\Gamma^{1:k}$ such that:

- $c_t^o = \emptyset \quad \forall t \in [0, J_{global}]$.

We can also interpret $\Gamma^{1:k}$ in another way:

- $c_t^o = \emptyset \quad \forall t, o$ and
- J_{global} is minimized.

where

$$c_t^o = \{J \mid \forall J \in [i, k] \text{ that are in conflict over the region } o \text{ at time } t\}.$$

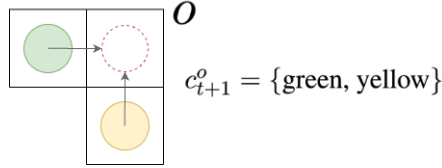


Figure 1: Explanation of collision c_t^o

4 MAPF Properties

MAPF has its own sub-community is because that it has its own taxonomy and its own internal set of structure properties. We will go through three of the most significant properties.

4.1 Optimality

Optimality in MAPF refers to minimizing the global cost J_{global} . With increasing computational power, speed is no longer the primary concern, making optimality a more critical factor from a practical perspective. Several approaches aim to maintain optimality, including K-agent A*, Operator Decomposition, Independent Detection, and M*.

Example: K-agent A*

Based on the scenario in part 2, we know that each agent in the system has a state space of 9 (the number of grids) and an action space of 5. If we have k agents in the system, there will be a state space of 9^k and an action space of 5^k .

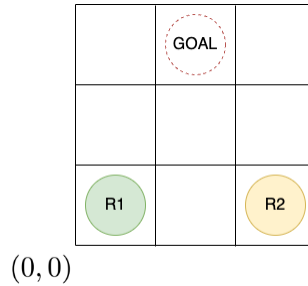


Figure 2: K-agent A* Scenario Cunstruction

Both state and action spaces are exponential in the number of agents. Therefore, this approach is optimal and ideal, but is not scalable. We cannot use it in robotics.

4.2 Fast

The second consideration after optimality is efficiency. We want a solver that runs quickly, ideally within polynomial time.

Example: Prioritized Planning

This approach assigns a priority order to the k agents, determining who moves first, second, and last. It runs A^* sequentially for each agent, ensuring a collision-free path since only one agent moves at a time while the others wait.

However, this method is far from optimal. With a large number of agents, such as a hundred, the sequential movement leads to significant inefficiencies, as later agents spend excessive time waiting instead of progressing toward their goals.

4.3 Completeness

The third property is completeness. A complete algorithm guarantees that if a solution exists, it will find it; otherwise, it will return failure.

5 Other Approaches

5.1 Operator Decomposition

Operator decomposition within Multi-Agent Pathfinding (MAPF) refers to breaking down the complex task of finding collision-free paths for multiple agents into simpler, manageable components or "operators." This helps to increase efficiency by decreasing the branching factor from 5^3 to a branching factor of 5 every intermediate step. The intuition for this is that we can run A^* independently on each operator and combine them resulting in A^* : $R^1 \times R^2 \times R^3$

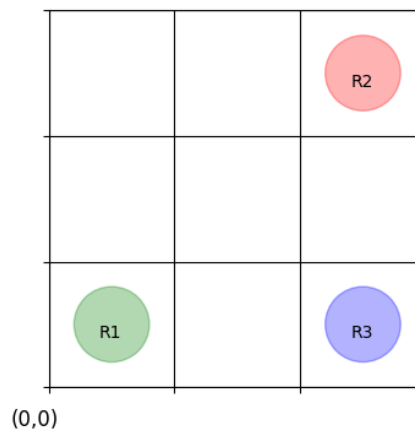
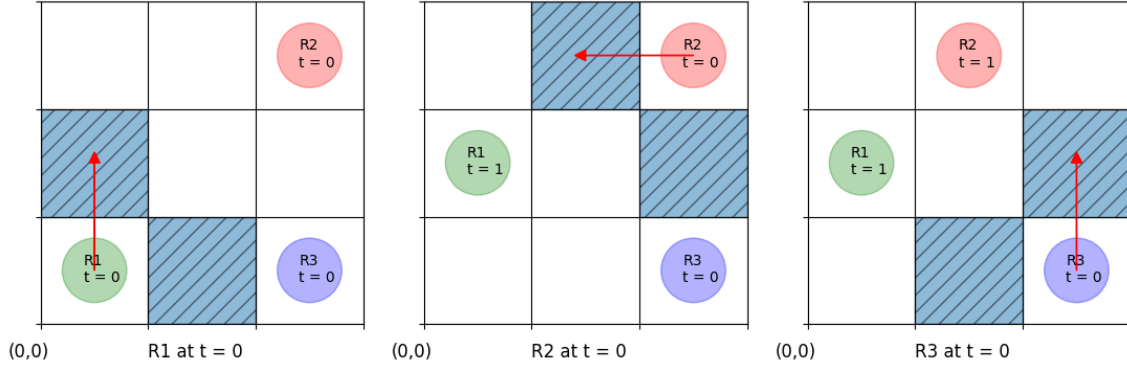


Figure 3: Operator Decomposition Intuition

Here we can see a 3x3 grid with our operators mapped onto it. Below shows how the algorithm steps through each operators potential path at $t = 0$ and how the graph updates as operators move across the graph.

$$\mathbf{A}^*: R^1 \times R^2 \times R^3$$



5.2 Independence Detection

The basic intuition behind Independence Detection is to start out by planning single agent paths, and then for any conflicting paths, clump them together and then internally solve them using $A^* + OD$

M*: Take a step back and explore each agent using A^* and then resolve conflicts as they arise.

5.3 Increasing Tree Cost Search (ITCS)

The key idea for ITCS is to treat MAPF as a look-up problem. As opposed to an optimization problem. This is done by creating a dictionary of all possible $\Gamma^{1:k}$ s and searching through this dictionary to find the solution that gets each agent to its goal in the least amount of steps.

Below is the definition for the dictionary of optimal costs.

c^i : optimal single-agent path length.

$$\langle c^0, c^1, \dots, c^k \rangle$$

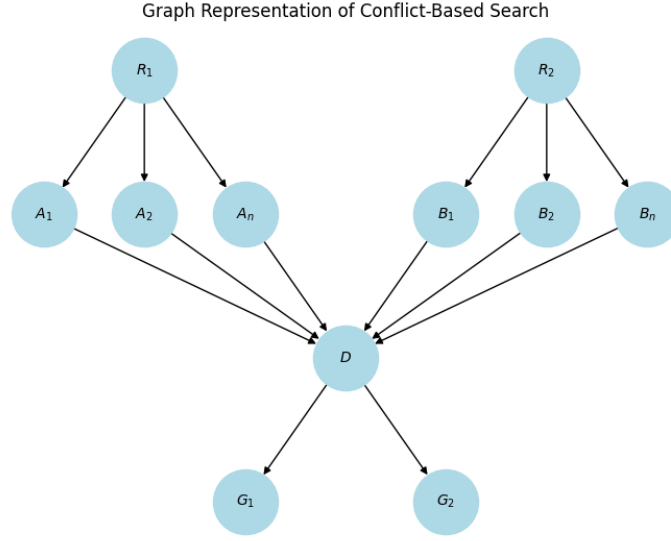
Here is the overall outline for the algorithm

At each step:

- Check for collision
 - If no collision, return node.
 - If collision:
 - * Increase $c^0 \leftarrow c^0 + 1$
 - Search through all agent 0 paths of length $c^0 + 1$. [Trivially, have agent 0 wait at hlc.] So number of paths to search linearly grows.
 - Repeat (increase the next agent's cost by 1) if again collision found.
- Breadth-first search

5.4 Conflict-Based Search

Conflict based search is another method similar to ICTS, but instead is best-first search. The algorithm first creates a conflict tree, seen below where the constraint is potential collisions along an agents path.



Using this tree we can then perform a low-level search given the constraints from the tree. And then each robots path can be recomputed optimally.

Equation Representation

At time $t = 0$:

$$R^1 = \{A_1, D, G_1\}$$

$$R^2 = \{B_1, D, G_2\}$$

Conflict:

$$C_0^{\{D\}} = \{R^1, R^2\}$$

Potential Resolutions:

$$R^1 = \{A_1, A_2, D, G_1\} \quad R^1 = \{A_1, D, G_1\}$$

$$R^2 = \{B_1, D, G_2\} \quad R^2 = \{B_1, B_2, D, G_2\}$$

R1 Waits

R2 Waits

5.5 ORCA-MAPF

The workspace for ORCA-MAPF is two-dimensional, operating in discrete time. A single interpreter can apply any velocity.

The change in position, Δx_t^i , for agent i at time t , is equivalent to the control input u_t^i which is also equivalent to the velocity v_t^i .

$$\Delta x_t^i = u_t^i = v_t^i \quad (1)$$

The absolute value of the velocity of agent i at time t is less than or equal to the maximum allowed velocity V_{max}^i .

$$|v_t^i| \leq V_{max}^i \quad (2)$$

This meaning that ORCA-MAPF does not require an optimal solution to obtain a result.

Algorithm

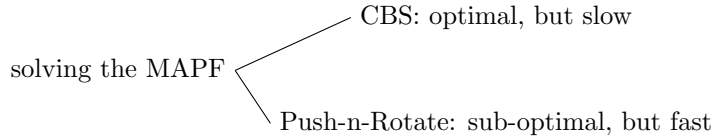
Each agent begins with an A* determined path that generates $\tilde{\Gamma}^i$

The agents continue to generate paths until they reach a deadlock [incorporate deadlock detection]

The default control policy is reprieved velocity obstacle c guarantees safety for single integrator systems

Deadlock Resolution

- Once you're in a deadlock, form a MAPF zone
 $\chi' \subseteq \chi$
- Determine start/goal steps, need a priority scheme



Set a timer: If CBS finishes before the timer, great. Otherwise select Push-n-Rotate

5.6 Social MAPF

The problem with the previous algorithm is that, while it is classified as a cooperative algorithm from a taxonomy perspective, it lacks fairness in practice. Specifically, individual agents perform very poorly.

As an alternative, using **auctions** enables agents to bid dynamically on paths or resources, promoting a more balanced distribution of effort and ensuring greater fairness.